

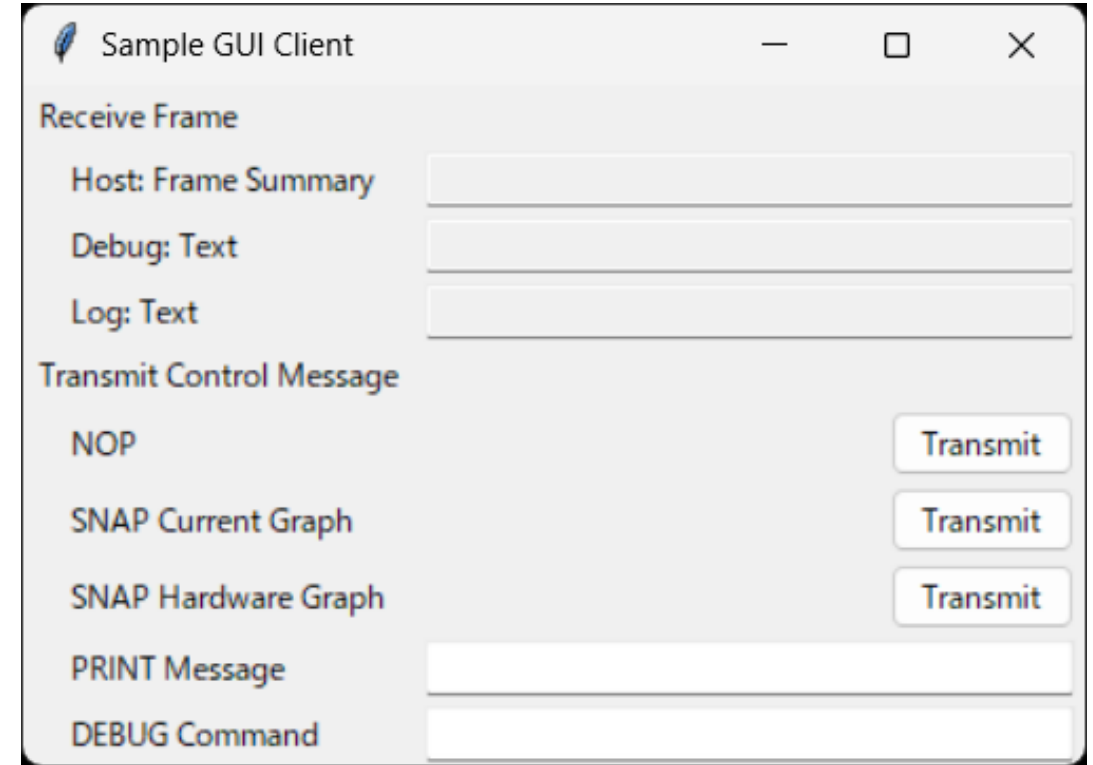
JIN 装置評価用アプリケーション Client アプリ開発者ガイド (α機 0.2.0)

2026/08/21

サンプルアプリ

```
C:\WINDOWS\system32\cmd
Ctrl-D で 終了します。
[PRINT] Last Serial Number = 100, Data Type = 0004h, Count = 100, Min = 21474
, Max = 2147400000, Median = [10747737.0, 2136673737.0]
[PRINT] Last Serial Number = 200, Data Type = 0004h, Count = 100, Min = 0, Ma
x = 2147464422, Median = [10726263.0, 2136587841.0]
[PRINT] Last Serial Number = 300, Data Type = 0004h, Count = 100, Min = 0, Ma
x = 2147464422, Median = [10726263.0, 2136501945.0]
[PRINT] Last Serial Number = 400, Data Type = 0004h, Count = 100, Min = 0, Ma
x = 2147464422, Median = [10726263.0, 2136416049.0]
[PRINT] Last Serial Number = 500, Data Type = 0004h, Count = 100, Min = 0, Ma
x = 2147464422, Median = [10726263.0, 2136330153.0]
[PRINT] Last Serial Number = 600, Data Type = 0004h, Count = 100, Min = 0, Ma
x = 2147464422, Median = [10726263.0, 2136244257.0]
[PRINT] Last Serial Number = 700, Data Type = 0004h, Count = 100, Min = 0, Ma
x = 2147464422, Median = [10726263.0, 2136158361.0]
[PRINT] Last Serial Number = 722, Data Type = 0004h, Count = 22, Min = 0, Max
= 2147464422, Median = [10726263.0, 461100465.0]
```

UI なし: SampleConsoleClient.py (199行)



UI あり: SampleGuiClient.py (335行)

共に Client アプリアーカイブ JinClients-0.2.0.MMDDn.zip に含まれる

Gateway アプリとの通信方式

Gateway アプリと Client アプリ間ではオープンソフトウェア ZeroMQ を用いたソケット通信でデータ受信、制御リクエスト送信のやり取りをする。

ZeroMQ の特徴

- 専用のサーバを必要としない。
- 高速で軽量のため、遅延が少ない。
- 1対1、1対多など多彩な通信パターンが用意されている。
- C++ の他、C#/Java/Javascript/Python のライブラリが存在する。

Python ラッパーライブラリ

- Client アプリアーカイブ JinClients-0.2.0.MMDDn.zip には ZeroMQ 特有の処理を意識せずに済むように Client アプリ向けに Python による専用ラッパークラス ZeroMQClient(.py) を用意している。

サンプルコード SampleConsoleClient.py

```
1  # Copyright 2026 Sony Global Manufacturing & Operations Corporation
2
3  import statistics
4  import struct
5  import sys
6  import threading
7  import time
8  from typing import List, Optional
9  from ZeroMQClient import ZeroMQClient
10
11 # ==== 設定 ====
12 SUB_ADDRESS: str = "tcp://localhost:55555"
13 PUSH_ADDRESS: str = "tcp://localhost:55556"
14
15 # ==== グローバル ====
16 running: bool = True
17 client: Optional[ZeroMQClient] = None
18
19 # =====
20 # 各トピックの処理関数 (カスタム部分)
21 #
22 def handle_host_topic(payload: bytes) -> None:
23     """
24     Host Port データの受信時処理
25
26     """
27
28
29 def handle_debug_topic(payload: str) -> None:
30     """ Debug トピック: 受信した文字列を即時 Print で Gateway に送信 """
31     client.send_print(f"Debug: {payload}",
32
33
34 def handle_log_topic(payload: str) -> None:
35     """ Log トピック: 受信した文字列を即時 Print で Gateway に送信 """
36     client.send_print(f"Log: {payload}", echo=True)
```

```
167 # =====
168 # メイン
169 # =====
170 def main() -> None:
171     global running, client
172     ラッパークラスインスタンスのセットアップ
173
174     # 汎用クライアントを生成し、トピックごとの処理関数を登録
175     client = ZeroMQClient(SUB_ADDRESS, PUSH_ADDRESS)
176     client.register_host_topic_handler(handle_host_topic)
177     client.register_debug_topic_handler(handle_debug_topic)
178     client.register_log_topic_handler(handle_log_topic)
179     client.start()
180
181     # Host 1秒集計スレッド起動
182     agg_thread = threading.Thread(target=host_aggregator_thread_func, daemon=True)
183     agg_thread.start()
184
185     try:
186         wait_ctrl_d()
187     except KeyboardInterrupt:
188         pass
189
190     # 終了処理
191     print("\n終了処理中...")
192     running = False
193     agg_thread.join(timeout=2.0)
194     client.stop()
195     print("終了しました。")
196
197 if __name__ == "__main__":
198     main()
199
```

補足1:

通信部分が関心事ではない場合、効率的に進めるために Client アプリアーカイブ JinClients-0.2.0.MMDDn.zip に含まれるサンプルアプリを編集して開始することを推奨する。

補足2:

前頁のサンプルでは Host Port, Debug Port, Log Port の受信コールバック3種類すべてを登録しているが、実際には Client アプリに必要なものだけ登録すればよい。

例えば、ラッパーライブラリ提供のメソッドのみで Debug Port, Log Port に関する必要なやりとりが実現できている MeasurementWizard.py や CommandPanel.py ではコールバックは何も登録していない。

受信処理

種別: SUB (localhost:55555)

メッセージ構成: フレーム数2 のマルチパート

- 第1フレーム = トピック
- 第2フレーム = データ

各トピックに対するデータの内容は下表の通り。

トピック名	データ
Host	Host Port の受信バイナリデータ本体 … 詳細は後述
Debug	Debug Port の受信テキスト1行分 (UTF-8, 改行コードは除去)
Log	Log Port の受信テキスト1行分 (UTF-8, 改行コードは除去)

Debug/Log Port データ受信コールバック

コールバックの型: callback(str) -> None

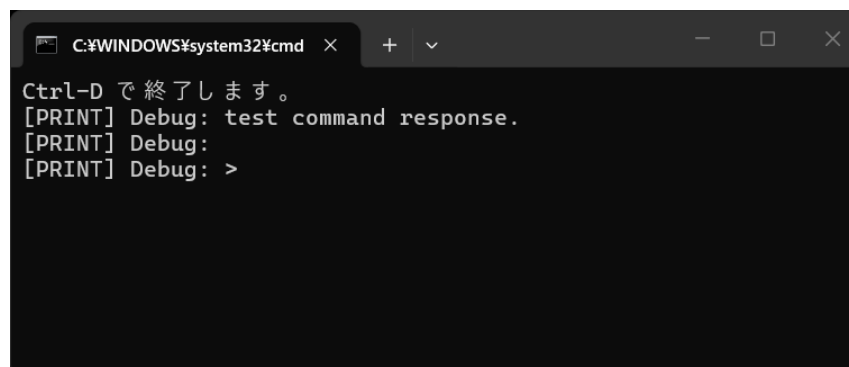
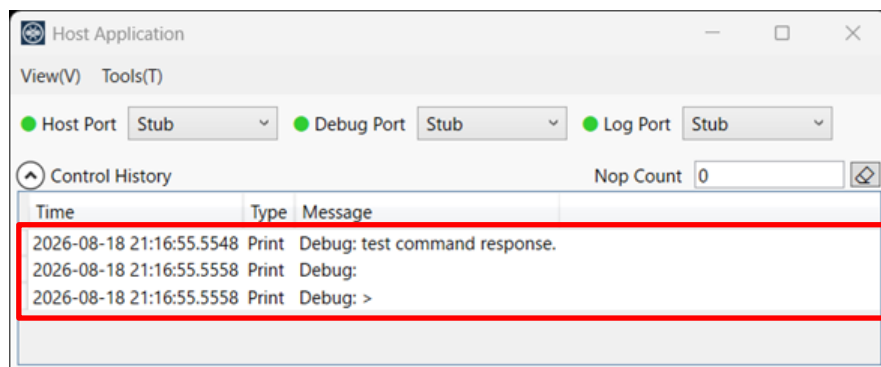
- 戻値 なし
- 第1引数 Debug/Log Port 受信 UTF-8 テキスト (1行)

サンプルコード SampleConsoleClient.py

サンプルコードの処理

1. 受信テキストの先頭に "Debug: " を追加
2. Gateway アプリに 1. のテキストの表示要求を送信

```
96  
97 def handle_debug_topic(payload: str) -> None:  
98     """ Debug トピック: 受信した文字列を即時 Print で Gateway に送信 """  
99     client.send_print(f"Debug: {payload}", echo=True)  
100  
101 def handle_log_topic(payload: str) -> None:  
102     """ Log トピック: 受信した文字列を即時 Print で Gateway に送信 """  
103     client.send_print(f"Log: {payload}", echo=True)  
104
```



Host Port データ受信コールバック

コールバックの型: callback(bytes) -> None

電流値の分析を行う場合、この実装が必要

- 戻値 なし
- 第1引数 Host Port 受信データ

サンプルコード SampleConsoleClient.py

サンプルコードの処理

1. [0:3] Serial Number 取得
2. [4:5] Data Type 取得
3. 電流値ありビーコン以外スキップ
4. [24:*] CurrentN の最大値、最小値、中央値を算出
5. 1秒分の最大値、最小値、中央値を算出
6. 1秒分蓄積したら結果を表示

```
31 def handle_host_topic(payload: bytes) -> None:
61     global host_last_receive_time, host_last_serial_number, host_last_data_type, host_total_max_value, host_total_min_value, host_median_list
62     if len(payload) >= 24:
63         serial_number = struct.unpack("<I", payload[0:4])[0]
64         data_type = struct.unpack("<H", payload[4:6])[0]
65
66         # データ種別 0001h-0004h 以外はスキップ
67         if data_type < 0x0001 or data_type > 0x0004:
68             return
69
70         # 24バイト目から4バイト×count個を符号付き32ビット整数として切り出して一括で展開
71         count = (len(payload) - 24) // 4
72         start_index = 24
73         end_index = start_index + (count * 4)
74         data_tuple = struct.unpack(f"<{count}i", payload[start_index:end_index])
75
76         # 最大値、最小値、中央値を算出
77         max_value = max(data_tuple)
78         min_value = min(data_tuple)
79         median_val = statistics.median(data_tuple)
80
81     with host_lock:
82         # 直近の受信とシリアル番号にギャップがある場合、またはデータ種別に変化がある場合はまず出力
83         if (host_last_serial_number != None) and ((serial_number != host_last_serial_number + 1) or (data_type != host_last_data_type)):
84             print_summary()
85
86         host_last_receive_time = time.monotonic()
87         host_last_serial_number = serial_number
88         host_last_data_type = data_type
89         host_total_max_value = max_value if host_total_max_value == None else max(host_total_max_value, max_value)
90         host_total_min_value = min_value if host_total_min_value == None else min(host_total_min_value, min_value)
91         host_median_list.append(median_val)
92
93         # 100 個 = 1秒分蓄積されていたら出力
94         if (len(host_median_list) == 100):
95             print_summary()
96
```

Host Port データフォーマット (1)

データフォーマット (共通)

	オフセット (バイト)	+0	+1	+2	+3	説明
共通 データ	0	Serial Number				[0:3] Serial Number … 装置の起動直後 0 から始まり、10ミリ秒ごとに +1 される数字（符号なし32ビット整数）。
	4	Data Type		Reserved		[4:5] Data Type … データ種別（次頁の一覧参照） [6:7] Reserved
個別 データ	8	Data Body				[8:~] Data Body … データ本体。データ種別ごとにフォーマットが異なる。
	...					

複数バイトに渡るデータはすべてリトルエンディアン。

Host Port データフォーマット (2)

データ種別一覧

分類	Data Type 値	説明
ビーコン	0000h	ビーコン ハードウェア値のみ
	0001h	ビーコン ハードウェア値 + ナノギャップ生成回路側 電流測定値 (10 kHz)
	0002h	ビーコン ハードウェア値 + 電流計測回路側 電流測定値 (10 kHz)
	0003h	ビーコン ハードウェア値 + 電流計測回路側 電流測定値 (50 kHz)
	0004h	ビーコン ハードウェア値 + 電流計測回路側 電流測定値 (100 kHz)
コマンド	0xxxh	予約 (各種コマンドレスポンス用途)
検査	FFFFh	検査用途

Host Port データフォーマット (3)

全ビーコン共通: データフォーマット個別データ部

	オフセット (バイト)	+0	+1	+2	+3	説明
共通データ	0	Serial Number				略
	4	Data Type		Reserved		略
個別データ	8	Bias	Reserved	+ep	-ep	[8] Bias … Bias 印可電圧。単位は 0.1V（符号なし8ビット整数） [10] +ep … 電気泳動印可電圧（+側）。単位は 0.1V（正值, 符号付き 8ビット整数） [11] -ep … 電気泳動印可電圧（-側）。単位は 0.1V（負値, 符号付き 8ビット整数）
	12	Current				[12:15] Current … 電流測定値（ 符号付き 32ビット整数）
	16	Motor Position				[16:19] Motor Position … ステッピングモーター位置。単位は 0.1um（ 符号付き 32ビット整数）
	20	Piezo Position				[20:23] Piezo Position … ピエゾアクチュエータ位置。単位は 1nm（ 符号付き 32ビット整数）
電流測定値	24	CurrentN				[24:~] CurrentN … 電流測定値。直近の 10ミリ秒間に測定した電流値を 符号付き 32ビット整数で時刻順に列挙 ＜次頁へ続く＞
	...					

Host Port データフォーマット (4)

全ビーコン共通: データフォーマット個別データ部 (続き)

	オフセット (バイト)	+0	+1	+2	+3	説明																		
個別データ	24	CurrentN				[24:~] CurrentN … 電流測定値。直近の 10 msec 間に測定した電流値を 符号付き 32ビット整数で時刻順に列挙																		
	...																							
<table><tr><th>Data Type</th><th>個数</th><th>総バイト数</th></tr><tr><td>0000h</td><td>0</td><td>0</td></tr><tr><td>0001h</td><td>100</td><td>400</td></tr><tr><td>0002h</td><td>100</td><td>400</td></tr><tr><td>0003h</td><td>500</td><td>2000</td></tr><tr><td>0004h</td><td>1000</td><td>4000</td></tr></table>							Data Type	個数	総バイト数	0000h	0	0	0001h	100	400	0002h	100	400	0003h	500	2000	0004h	1000	4000
Data Type	個数	総バイト数																						
0000h	0	0																						
0001h	100	400																						
0002h	100	400																						
0003h	500	2000																						
0004h	1000	4000																						

電流測定値は測定回路の AD コンバータのレジスタ値であり、下表に従い、アンペアに変換できる。

F/W バージョン	精度 (回路)	Bias 印可電圧	変換式
0.2.0	Low (ナノギャップ生成回路)	0.1V	電流値 [uA] = レジスタ値 / 42781900.799999997
		0.2V ~	非サポート
	High (電流計測回路)	0.1V	電流値 [pA] = レジスタ値 / 5825183.6129280003
		0.2V ~	非サポート

送信処理

送信ソケット

種別: PUSH (localhost:55556)

メッセージ構成: フレーム数1のシングルパート

データフォーマット:

	オフセット (バイト)	名称	説明
共通データ	0	Control Type	[0] Control Type … コントロール種別
個別データ	1	Data Body	[1:*] Data Body … データ本体。コントロール種別ごとにフォーマットが異なる。
	...		

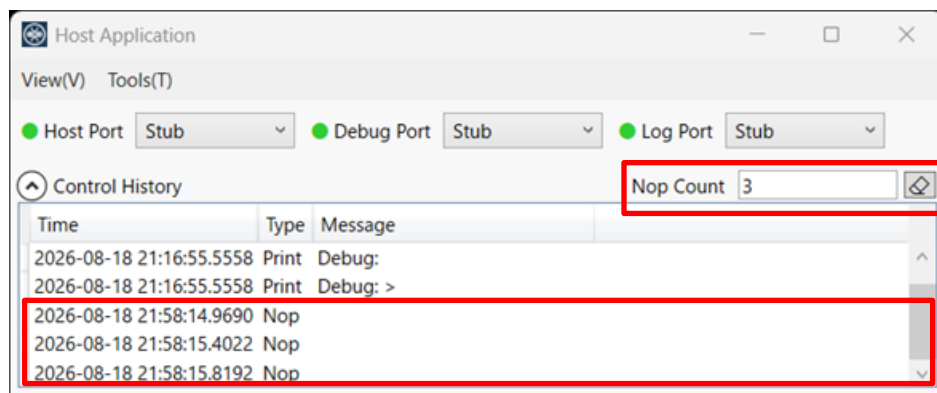
各コントロール種別に対する Gateway アプリへの要求及びデータの内容は下表の通り。

値	名称	Gateway アプリへの要求	データ
00h	Nop	何もしない（デバッグ用途）。	なし
01h	SnapCurrentGraph	電流値グラフのスナップを取得する。	なし
02h	SnapHardwareGraph	ハードウェア値グラフのスナップを取得する。	なし
80h	Print	送信したテキストを Gateway アプリの記録に残す。	テキスト1行分（UTF-8, 改行コードなし）
FEh	Debug	送信したテキストを Debug Port へ転送する。	テキスト1行分（UTF-8, 改行コードなし）
FFh	Host	送信したバイナリデータを Host Port へ転送する。	バイナリデータ

Nop

要求動作: 何もしない（通信状態の確認等、デバッグ用途）

送信すると Gateway アプリのメイン画面の「Nop Count」の数が +1 ずつ増加する。

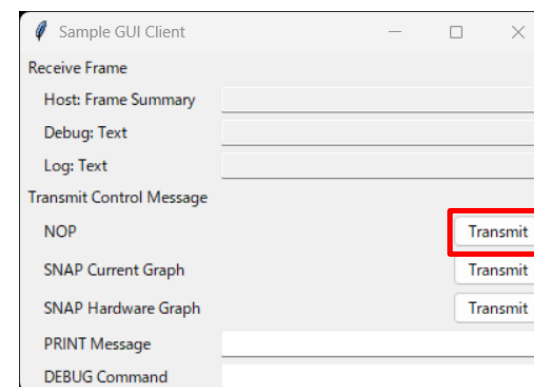


クラス ZeroMQClient メソッド

- `def send_nop(self) -> None:`

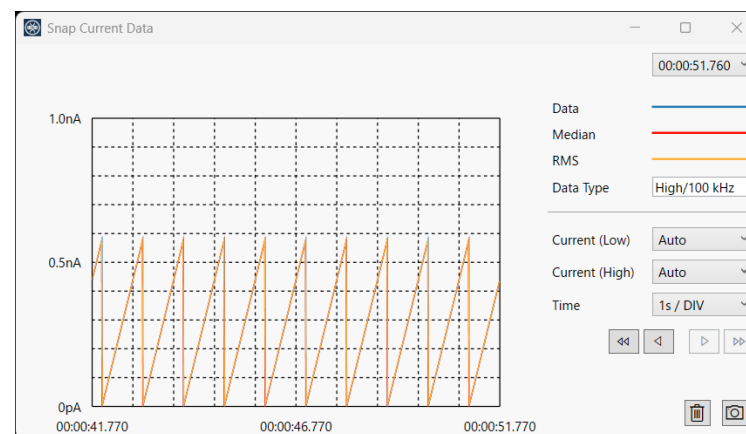
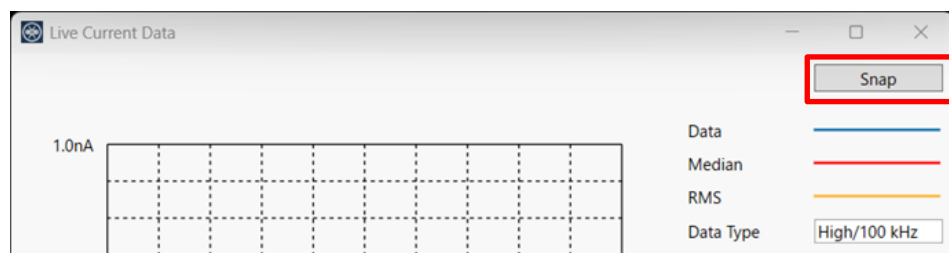
サンプルコード SampleGuiClient.py

```
151  
152     def _on_nop_clicked(self) -> None:  
153         if client is not None:  
154             client.send_nop()  
155
```



SnapCurrentGraph

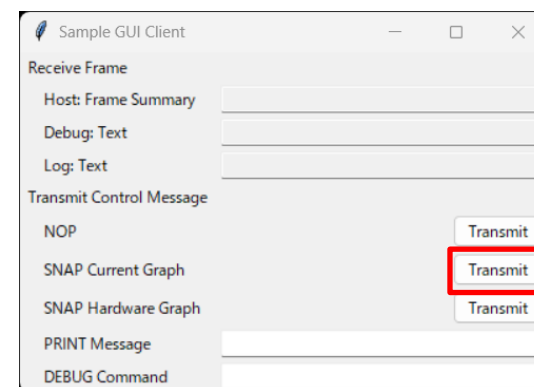
要求動作: Gateway アプリで現在の電流値グラフのスナップを取得する。
送信するとライブ画面の「Snap」ボタン押下と同じ動作をする。



クラス ZeroMQClient メソッド

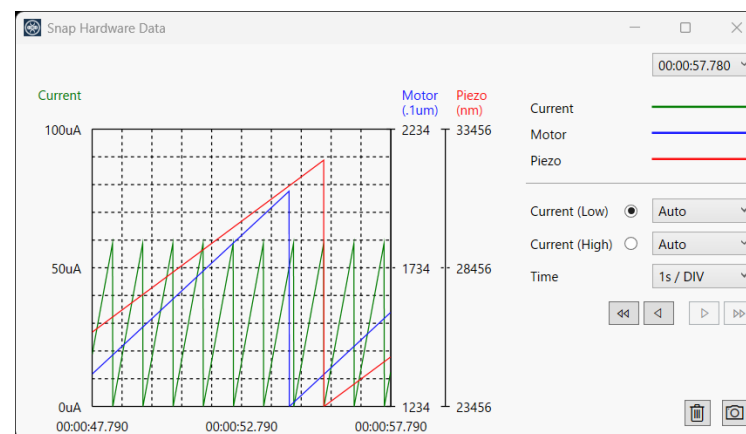
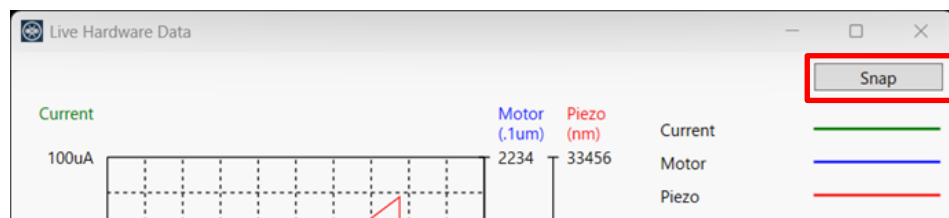
- `def send_snap_current_graph(self) -> None:`
サンプルコード SampleGuiClient.py

```
155  
156     def _on_snap_current_graph_clicked(self) -> None:  
157         if client is not None:  
158             client.send_snap_current_graph()  
159
```



SnapHardwareGraph

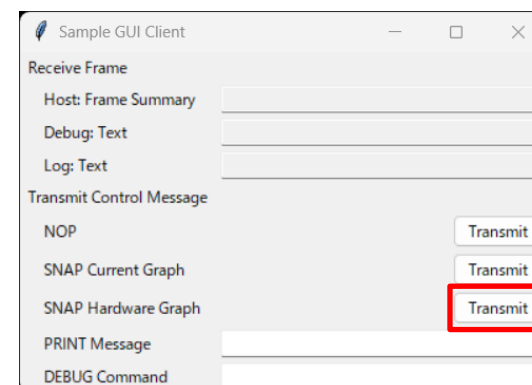
要求動作: Gateway アプリで現在のハードウェア値グラフのスナップを取得する。
送信するとライブ画面の「Snap」ボタン押下と同じ動作をする。



クラス ZeroMQClient メソッド

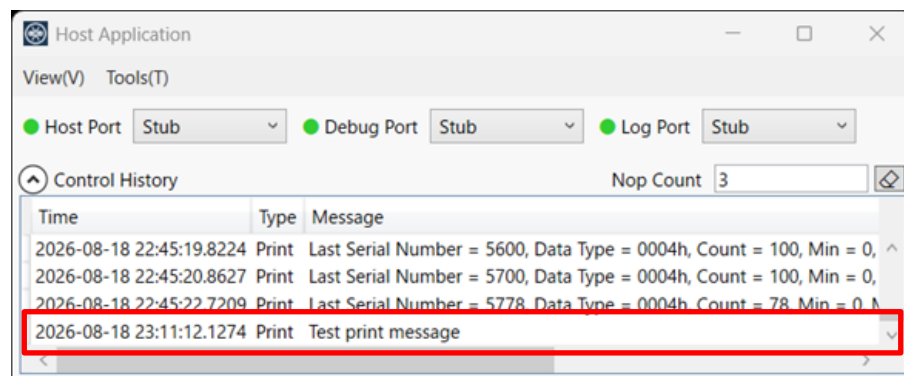
- `def send_snap_hardware_graph(self) -> None:`
サンプルコード SampleGuiClient.py

```
159
160     def _on_snap_hardware_graph_clicked(self) -> None:
161         if client is not None:
162             client.send_snap_hardware_graph()
163
```



要求動作: 指定のテキストを Gateway アプリの記録に残す。

送信したテキストが Gateway アプリのメイン画面のコントロール履歴に表示されると共に、ログファイルに記録される。

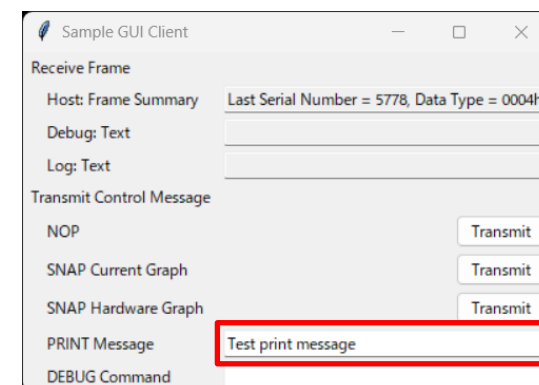


クラス ZeroMQClient メソッド

- `def send_print(self, text: str, echo: bool = False) -> None:`

サンプルコード SampleGuiClient.py

```
137 def _on_print_entered(self, event: tk.Event) -> None:
138     text = self.var_print_input.get()
139     if text and client is not None:
140         client.send_print(text)
```



要求動作: 指定のテキストを Debug Port へ転送する。

送信したテキストを Debug Port に送信し、コマンドとして実行する。

注意: 装置側は Debug Port コマンドを受け取ると、その実行が完了するまで、次のコマンドは受け付けない。
実行完了は Debug Port からのテキスト ">" の受信をもって判断する必要がある。

クラス ZeroMQClient メソッド

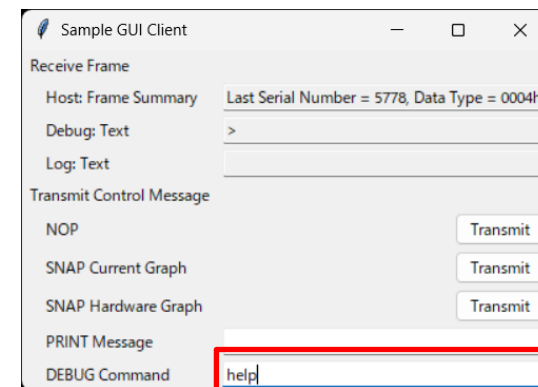
- `async def send_debug_async(self, text: str) -> List[str]:` **【非推奨】**
await 付きで呼び出すとテキスト ">" の受信を待機し、それまで受信したすべての行を List にして返す。

非推奨の背景:

Debug Port コマンドは今後の F/W で変更、または Host Port へ移動する可能性がある。そのため、コマンドごとにメソッドが用意されており（後述）、そちらを推奨する。

サンプルコード SampleGuiClient.py

```
141
142     def _on_debug_entered(self, event: tk.Event) -> None:
143         text = self.var_debug_input.get()
144         if text and client is not None:
145             asyncio.run_coroutine_threadsafe(self._send_debug_async(text), self.async_loop)
146
147     async def _send_debug_async(self, text: str) -> None:
148         self.root.after(0, lambda: self.entry_debug_in.config(state=tk.DISABLED))
149         await client.send_debug_async(text)
150         self.root.after(0, lambda: self.entry_debug_in.config(state=tk.NORMAL))
151
```



Debug Port コマンド実行メソッド一覧 (1)

メソッド	対応する Debug Port コマンド
async def command_set_bias_async(self, bias: int) -> bool:	dd_ep bias [bias]
async def command_set_ep_async(self, ep: int) -> bool:	dd_ep ep [ep]
async def command_move_to_zero_point_async(self) -> bool:	mw_ac go0
async def command_start_sampling_low_10k_async(self) -> bool:	sv_info_sender start 0
async def command_start_sampling_high_10k_async(self) -> bool:	sv_info_sender start 10
async def command_start_sampling_high_50k_async(self) -> bool:	sv_info_sender start 2
async def command_start_sampling_high_100k_async(self) -> bool:	sv_info_sender start 1
async def command_stop_sampling_async(self) -> bool:	sv_info_sender stop
async def command_mcbj_stop_async(self) -> bool:	mcbj stop
async def command_mcbj_first_cut_async(self) -> bool:	mcbj fc start
async def command_mcbj_targeting_async(self) -> bool:	mcbj targeting start
async def command_mcbj_motor_training_async(self) -> bool:	mcbj mt start
async def command_mcbj_piezo_training_async(self) -> bool:	mcbj pt start
async def command_mcbj_auto_cut_async(self) -> bool:	mcbj ac start
async def command_mcbj_calibration_async(self) -> bool:	mcbj cal start
async def command_asz_expand_gap_async(self) -> bool:	asz hg start
async def command_asz_hold_gap_async(self) -> bool:	asz eg start

戻値はすべて成功時 True、エラー時 False。

Debug Port コマンド実行メソッド一覧 (2)

メソッド	対応する Debug Port コマンド
<code>async def command_mcbj_set_fc_up_limit_async(self, up_limit_displacement: int) -> bool:</code>	<code>mcbj set fc up_limit [up_limit_displacement]</code>
<code>async def command_mcbj_set_fc_last_down_distance_async(self, distance: int) -> bool:</code>	<code>mcbj set fc last_down_distance [distance]</code>
<code>async def command_mcbj_set_fc_threshold_async(self, threshold_current: int) -> bool:</code>	<code>mcbj set fc threshold [threshold_current]</code>
<code>async def command_mcbj_set_fc_speed_table_async(self, trial_index: int, up_speed: int, down_speed: int) -> bool:</code>	<code>mcbj set fc speed_table [trial_index] [up_speed] [down_speed]</code>
<code>async def command_mcbj_set_mt_async(self, lower_limit_current: int, upper_limit_current: int, motor_speed: int) -> bool:</code>	<code>mcbj set mt ROUGH [lower_limit_current] [upper_limit_current] [motor_speed]</code>
<code>async def command_mcbj_set_pt_async(self, lower_limit_current: int, upper_limit_current: int, piezo_speed: int) -> bool:</code>	<code>mcbj set pt SMOOTH [lower_limit_current] [upper_limit_current] [piezo_speed]</code>
<code>async def command_mcbj_set_targeting_threshold_async(self, threshold_current: int) -> bool:</code>	<code>mcbj set targeting threshold [threshold_current]</code>
<code>async def command_mcbj_set_targeting_up_table_async(self, index: int, upper_limit_current: int, displacement: int, speed: int) -> bool:</code>	<code>mcbj set targeting up_table [index] [upper_limit_current] [displacement] [speed]</code>
<code>async def command_mcbj_set_targeting_down_table_async(self, index: int, upper_limit_current: int, displacement: int, speed: int) -> bool:</code>	<code>mcbj set targeting down_table [index] [upper_limit_current] [displacement] [speed]</code>
<code>async def command_mcbj_set_targeting_up_limit_async(self, up_limit_displacement: int) -> bool:</code>	<code>mcbj set targeting up_limit [up_limit_displacement]</code>
<code>async def command_mcbj_set_targeting_timeout_async(self, timeout_milliseconds: int) -> bool:</code>	<code>mcbj set targeting timeout [timeout_milliseconds]</code>

戻値はすべて成功時 True、エラー時 False。

Debug Port コマンド実行メソッド一覧 (3)

メソッド	対応する Debug Port コマンド
<code>async def command_mcbj_set_ac_threshold_async(self, threshold_current: int) -> bool:</code>	<code>mcbj set ac threshold [threshold_current]</code>
<code>async def command_mcbj_set_ac_up_table_async(self, index: int, upper_limit_current: int, displacement: int, speed: int) -> bool:</code>	<code>mcbj set ac up_table [index] [upper_limit_current] [displacement] [speed]</code>
<code>async def command_set_ac_timeout_async(self, timeout_milliseconds: int) -> bool:</code>	<code>mcbj set ac timeout [timeout_milliseconds]</code>
<code>async def command_mcbj_set_cal_limit_async(self, lower_limit_current: int, upper_limit_current: int) -> bool:</code>	<code>mcbj set cal limit [lower_limit_current] [upper_limit_current]</code>
<code>async def command_mcbj_set_cal_speed_async(self, speed: int) -> bool:</code>	<code>mcbj set cal speed [speed]</code>
<code>async def command_asz_set_eg_delta_async(self, delta: int) -> bool:</code>	<code>asz set eg delta [delta]</code>
<code>async def command_asz_set_eg_stable_current_range_async(self, range_current: int) -> bool:</code>	<code>asz set eg stable_current_range [range_current]</code>
<code>async def command_asz_set_eg_stable_current_count_async(self, count: int) -> bool:</code>	<code>asz set eg stable_current_count [count]</code>
<code>async def command_asz_set_hg_tunnel_current_async(self, tunnel_current: int) -> bool:</code>	<code>asz set hg tunnel_current [tunnel_current]</code>
<code>async def command_asz_set_hg_fb_table_async(self, index: int, upper_limit_gap_current: int, displacement: int, speed: int) -> bool:</code>	<code>asz set hg fb_table [index] [upper_limit_gap_current] [displacement] [speed]</code>

戻値はすべて成功時 True、エラー時 False。

Host

要求動作: 指定のバイナリデータを Host Port へ転送する。

送信したバイナリデータを Host Port に送信し、コマンドとして実行する。

ファームウェア 0.2.0 では Host Port コマンドは未実装のため、送信しても何も発生しない。

変更履歴

更新日	対象バージョン	内容
2026/08/21	0.2.0	新規作成

SONY

SONYはソニー株式会社の登録商標または商標です。

各ソニー製品の商品名・サービス名はソニー株式会社またはグループ各社の登録商標または商標です。その他の製品および会社名は、各社の商号、登録商標または商標です。